

To ensure that the *minions* have the new *pillar data*, issue a command to them asking that they fetch their *pillars* from the *master*:

```
salt '*' saltutil.refresh_pillar
```

Like *grains*, *pillars* can also be used in the *cmd* line. The syntax is as follows:

```
salt -I 'somekey:specialvalue' test.ping
salt -I 'master:id:myminion' pillar.items
```

Targeting

Remote execution is one of the core features in Salt, and we can achieve most of the tasks through this as it saves a lot of time. The syntax of the command line execution is as follows:

```
salt <target> <function> [arguments]
```

The target component allows you to filter which minions should run the following function. The default filter is a *glob* on the *minion id*.

```
salt '*' test.ping
salt 'machine_name' test.ping
salt '*.test.com' test.ping (we can use arbitrary or regular expression in targets)
```

Grains and pillars can be used to specify targets:

```
salt -G 'os:redhat' test.ping
salt -I 'master:id:myminion' pillar.items
```

An example of multiple target types:

```
salt -C '@os:redhat or I@master:id:myminion' test.ping
```

Here are some shell commands:

```
salt '*' cmd.run 'uname -a'
salt '*' cmd.run 'rpm -qa|grep vim'
```

Examples of space-delimited arguments:

```
salt '*' cmd.exec_code python 'import sys; print sys.version'
salt '*' cmd.exec_code bash 'echo ${seq 1 5}'
```

The render/template system

SLS data doesn't need to be represented in *YAML*. Salt defaults to *YAML* because it is simple, straightforward and easy to learn. The default rendering system is the *YAML+ Jinja* renderer. The *YAML+ Jinja* renderer will first pass the

template through the *Jinja2* templating system and then through the *YAML* parser. The benefit here is that full programming constructs are available when creating *SLS* files.

The templating system is configured through the *renderer* value in the master *config* file. Managed files and dynamic content can be rendered through templates. In *SLS* data you should pass the rendering system in the template variable.

The supported formats are:

- *Jinja + YAML*
- *Mako + YAML*
- *Wempy + YAML*
- *Jinja + json*
- *Mako + json*
- *Wempy + json*

Here's an example of the *motd* file:

```
/etc/motd:
file.managed:
{% if grains['os'] == 'RedHat' %}
- source: salt://motd/files/motd.rhel
{% elif grains['os'] == 'Ubuntu' %}
- source: salt://base/motd/files/motd.ubuntu
{% endif %}
- force: True
```

In the above *.sls* file, the *file.managed* state uses the *Jinja* templating system to generate different messages for Red Hat and Ubuntu OSs, using an *if* statement.

Here's how you can use iterations (for-loops):

```
{% for usr in ['john','henry','pal'] %}
{{ usr }}:
user.present
{% endfor %}
```

An example of pillar data in templates:

```
/srv/pillar/users/init.sls
users:
john: 1000
henry: 1001
pal: 1002
```

Update the *top.sls* file of the pillar as follows:

```
/srv/pillar/top.sls:
base:
- '*'
- pkgs
- users
```

Refresh the pillar data to ensure users are available for all *minions* as follows:

...Continued on page 68